Redes (TA048)
TP N°1: File Transfer

Esteban Carisimo y Juan Ignacio Lopez Pecora

Facultad de Ingeniería, Universidad de Buenos Aires

10 de septiembre de 2026

Resumen

El presente trabajo práctico tiene como objetivo la creación de una aplicación de red. Para tal finalidad, será necesario comprender cómo se comunican los procesos a través de la red, y cuál es el modelo de servicio que la capa de transporte le ofrece a la capa de aplicación. Además, para poder lograr el objetivo planteado, se aprenderá el uso de la interfaz de sockets y los principios básicos de la transferencia de datos confiable (del inglés Reliable Data Transfer, RDT).

Palabras clave— Socket, protocolo, tcp, udp

1. Propuesta de trabajo

Este trabajo práctico se plantea como objetivo la comprensión y la puesta en práctica de los conceptos y herramientas necesarias para la implementación de un protocolo RDT. Para lograr este objetivo, se deberá desarrollar una aplicación de arquitectura cliente-servidor que implemente la funcionalidad de transferencia de archivos mediante las siguientes operaciones:

- **UPLOAD**: Transferencia de un archivo del cliente hacia el servidor
- **DOWNLOAD**: Transferencia de un archivo del servidor hacia el cliente

Dada las diferentes operaciones que pueden realizarse entre el cliente y el servidor, se requiere del diseño e implementación de un protocolo de aplicación básico que especifique los mensajes intercambiados entre los distintos procesos.

2. Herramientas a utilizar y procedimientos

La implementación de las aplicaciones solicitadas deben cumplir los siguientes requisitos:

- La aplicaciones deben ser desarrolladas en lenguaje Python [1] utilizando la librería estandard de sockets [2].
- La comunicación entre los procesos se debe implementar utilizando UDP como protocolo de capa de transporte.
- Las aplicaciones cliente/servidor deben ser desplegadas en una topología simulada con *mininet*.
- Para lograr una transferencia confiable al utilizar el protocolo UDP, se pide implementar una versión utilizando el protocolo *Stop & Wait* y otra versión utilizando el mecanismo de error-recovery de *Selective Repeat*.

Para poder validar que el protocolo desarrollado provee garantía de entrega es necesario forzar la pérdida de paquetes. Para poder simular distintas condiciones de red se pide utilizar la herramienta `mininet`.

3. Requisitos críticos de implementación

La implementación de las aplicaciones solicitadas debe cumplir los siguientes requisitos:

- Se debe poder cargar/descargar archivos binarios.
- El protocolo debe garantizar la entrega de paquetes para una pérdida del 10 % sobre los enlaces (tasa total del camino en cada sentido).
- El protocolo deberá poder operar correctamente con valores de RTT de hasta 300 ms.
- Las operaciones de carga/descarga deben completarse en menos de 2 minutos para archivos de prueba de tamaño 5 MB (usando el protocolo con *Selective Repeat*, y con la red configurada a un 10 % de pérdida y un RTT de 40 ms).
- La implementación deberá evitar retransmisiones excesivas ante pérdidas, procurando minimizar el tráfico adicional generado por la transferencia. Esto podrá ser verificado mediante una captura de tráfico con *Wireshark*.
- La aplicación debe contemplar por lo menos 2 condiciones de error.
- El servidor debe ser capaz de procesar de manera concurrente la transferencia de archivos con múltiples clientes utilizando diferentes protocolos.
- El protocolo de capa de aplicación debe contemplar flujo de errores.

3.1. Interfaz del cliente

La funcionalidad del cliente se divide en dos aplicaciónes de línea de comandos: `upload` y `download`. El comando `upload` envía un archivo al servidor para ser guardado con el nombre asignado. El listado 1 especifica la interfaz de línea de comandos para la operación de upload:

```
> python upload -h
usage: upload [-h] [-v | -q] [-H ADDR] [-p PORT] [-s FILEPATH] [-n FILENAME] [-r
    protocol]

<command description>

optional arguments:
  -h, --help     show this help message and exit
  -v, --verbose  increase output verbosity
  -q, --quiet    decrease output verbosity
  -H, --host     server IP address
  -p, --port     server port
  -s, --src      source file path
  -n, --name     file name
  -r, --protocol error recovery protocol
```

Listing 1: Interfaz de linea de comandos de upload

El comando `download` descarga un archivo especificado desde el servidor. El listado 2 especifica la interfaz de linea de comandos para la operación de download:

```
> python download -h
usage: download [-h] [-v | -q] [-H ADDR] [-p PORT] [-d FILEPATH] [-n FILENAME] [-r
    protocol]

<command description>

optional arguments:
  -h, --help     show this help message and exit
  -v, --verbose  increase output verbosity
  -q, --quiet    decrease output verbosity
  -H, --host     server IP address
  -p, --port     server port
  -d, --dst      destination file path
  -n, --name     file name
  -r, --protocol error recovery protocol
```

Listing 2: Interfaz de linea de comandos de download

3.2. Interfaz del servidor

El servidor provee el servicio de almacenamiento y descarga de archivos. El listado 3 especifica la interfaz de linea de comandos para el inicio del servidor:

```
> python start-server -h
usage: start-server [-h] [-v | -q] [-H ADDR] [-p PORT] [-s DIRPATH]

<command description>

optional arguments:
  -h, --help     show this help message and exit
  -v, --verbose  increase output verbosity
  -q, --quiet    decrease output verbosity
  -H, --host     service IP address
  -p, --port     service port
  -s, --storage  storage dir path
```

Listing 3: Interfaz de linea de comandos del servidor

4. Análisis

Comparar el tiempo necesario para enviar archivos entre la versión *Selective Repeat* del protocolo y la versión *Stop&Wait*, utilizando archivos de distintos tamaños (al menos 3) y bajo distintas configuraciones de pérdida de paquetes.

Calcule el *throughput* promedio para cada configuración.

5. Preguntas a responder

1. Describa la arquitectura Cliente-Servidor.
2. ¿Cuál es la función de un protocolo de capa de aplicación?

3. Detalle el protocolo de aplicación desarrollado en este trabajo.

4. La capa de transporte del stack TCP/IP ofrece dos protocolos: TCP y UDP. ¿Qué servicios proveen dichos protocolos? ¿Cuáles son sus características? ¿Cuándo es apropiado utilizar cada uno?

5. Justifique si el protocolo desarrollado cuenta con mecanismos de control de congestión, en caso de tenerlos, descríbalos.

6. ¿Cuál de los dos protocolos desarrollados envía archivos en la menor cantidad de tiempo? ¿Es siempre el mismo?

6. Entrega

La entrega consta de un informe, el código fuente de la aplicación desarrollada y un archivo README en formato MarkDown[3] con los detalles necesarios para ejecutar la aplicación. La codificación debe cumplir el standard PEP8 [4], para ello se sugiere utilizar el linter flake8 [5]. La figure 1 muestra la estructura y el contenido del archivo ZIP entregable.

```
tp2.zip
├── informe.pdf
├── src/
│   ├── lib/
│   ├── upload
│   ├── download
│   └── start-server
└── README.md
```

Figura 1: Estructura del archivo zip entregable

6.1. Fecha de entrega

La entrega se hará a través del campus. La fecha de entrega está pautada para el día jueves 08 de Octubre de 2026 a las 19.00hs. **Cualquier entrega fuera de término no será considerada.**

6.2. Informe

La entrega debe contar con un informe donde se demuestre conocimiento de la interfaz de sockets, así como también los resultados de las ejecuciones de prueba (capturas de ejecución de cliente y logs del servidor). El informe debe describir la arquitectura de la aplicación. En particular, se pide detallar el protocolo de red implementado para cada una de las operaciones requeridas.

El informe debe tener la siguientes secciones:

- Introducción
- Hipótesis y suposiciones realizadas
- Implementación
- Pruebas
- Preguntas a responder
- Dificultades encontradas
- Conclusión

Referencias

[1] Guido Van Rossum and Fred L. Drake. *Python 3 Reference Manual*. CreateSpace, Scotts Valley, CA, 2009.

[2] Python Software Foundation. *socket — Low-level networking interface*, 2020. `https://docs.python.org/3/library/socket.html` [Accessed: 15/10/2020].

[3] John Gruber. *Markdown*, 2020. `https://daringfireball.net/projects/markdown/` [Accessed: 15/10/2020].

[4] Guido van Rossum, Barry Warsaw, and Nick Coghlan. Style guide for Python code. PEP 8, Python Software Foundation, 2001.

[5] Ian Stapleton Cordasco. *Flake8: Your Tool For Style Guide Enforcement*, 2020. `https://flake8.pycqa.org/en/latest/` [Accessed: 15/10/2020].